Strings Meet Lists, Lists Meet Strings

ISTA 130

Last Time

➔ Python list data type:

- Operators, functions
- Traversing
- Mutability
- Methods

```
>>> my_list = [1, 2, 3]
>>> my_list.append('a')
>>> my_list
[1, 2, 3, 'a']
>>> my_list.insert(0, 'b')
>>> my_list
['b', 1, 2, 3, 'a']
>>> my_list.pop()
'a'
>>> my_list
['b', 1, 2, 3]
>>> my_list.pop(2)
2
>>> my_list
['b', 1, 3]
```

String Method: `split`

- `str.split()`

  Return a *list* of the words in the string `str`. The string is separated on whitespaces and empty strings are removed from the result.

```
phrase = "Once upon a time there lived a rabbit."
words = phrase.split()
print(words) →
['Once', 'upon', 'a', 'time', 'there', 'lived',
'a', 'rabbit.']
```

String Method: `split`

You can also specify where to make the splits:

- `str.split(separator)`

  Return a *list* of the words in the string `str`, using `separator` as the delimiter string.

```
time = '1:43:26'
hour_min_sec = time.split(':')
print(hour_min_sec) →
['1', '43', '26']
```

- Notice that the delimiter is not found in the returned list.

`split`

- `split` is a string method – call it on a string using the dot operator.
- Called without an argument, it splits on all whitespace – tabs, spaces, and newlines.
- Optionally pass it a string to split on instead of whitespace.
- It always returns a list of strings.

Converting Lists of strings to Strings

- You may need to go the other direction: convert a *list of strings* into a single string.
- You can do this with the string **join** *method*:

```
words = ['Once', 'upon', 'a', 'time', 'there',
'lived', 'a', 'rabbit.']
phrase = ' '.join(words)
print(phrase)
→ 'Once upon a time there lived a rabbit.'
```

- `join` is ***string*** method
- *You must **call it on a string** and **pass it a list of strings**.*

Converting Lists to Strings

- The string that calls `join` is the "glue" that all of the strings in the list are stuck together with:

```
hour_min_sec = ['1', '43', '26']
time = ':'.join(hour_min_sec)
print(time)  → 1:43:26
```

- Read as:
  - "use colons to join all **strings** in the list `hour_min_sec`"

Strings to Lists and Vice Versa

- Converting strings to list with the `split` string method and lists to strings with the `join` string method is a critical Python skill.

Lists as Arguments

- You can, of course, pass lists as arguments to functions
- Is it surprising that the list object can be modified inside the function?
- We've been doing it with turtles all along:

Mutability and Assignment to a Position

- Assignment to a variable does **nothing** to the object a variable refers to.

```
stuff = [1, 2, 3]
stuff = 'banana'
```

- Assignment to an indexed location in a mutable sequence *does* alter the object referred to by the variable

```
stuff = [1, 2, 3]
stuff[0] = 'yams'
```

  - Location (or 'slot') 0 in the list referred to by `stuff` now refers to a different object.
  - Think of **`stuff[0] = 'yams'`** as just a shortcut for:

```
stuff.pop(0)
stuff.insert(0, 'yams')
```

Mutable vs. Immutable

- Mutable objects can be changed by assigning to a position in the object.
- Immutable objects cannot – Python won't let you. Are strings mutable or immutable?
- Unfortunately, mutable and immutable sound a lot alike when pronounced, so pay attention to context.

Mutability

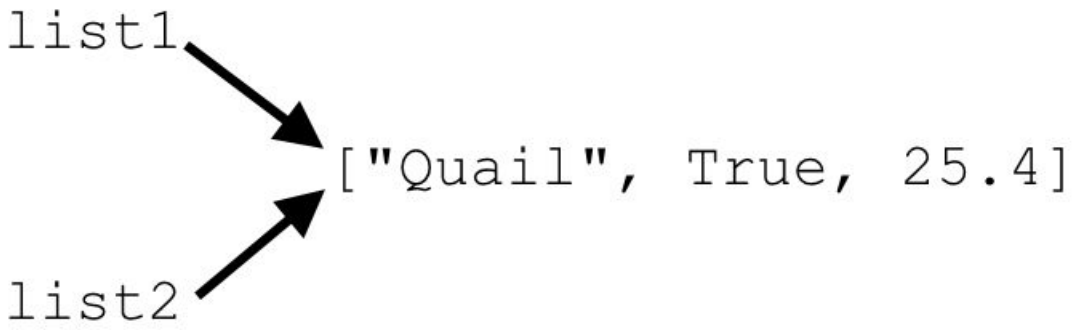
- Mutable types often have some methods that mutate the object and other methods that don't:

```
some_list.pop() # removes an element
some_list.count(value) # just returns an int
```

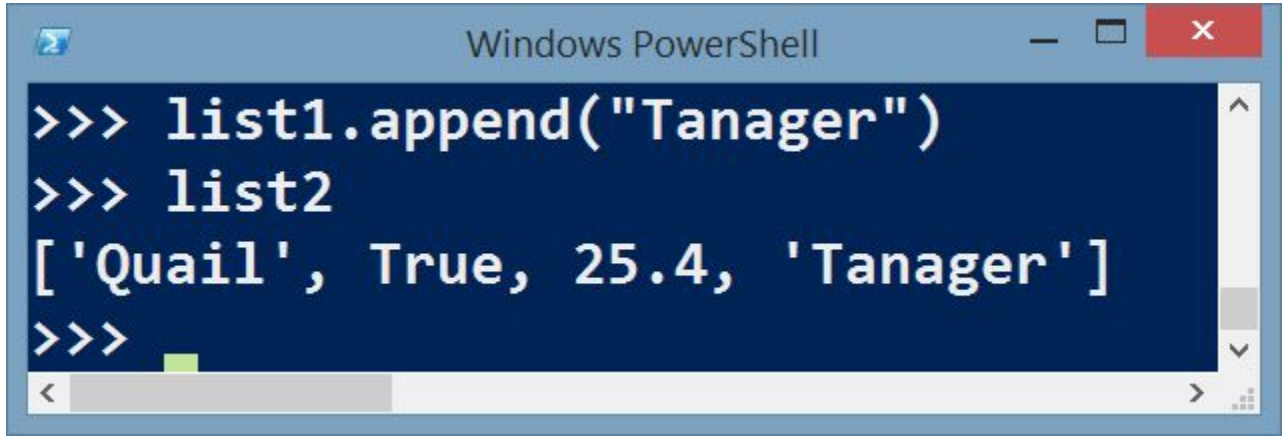
This gives us another way to get information out of functions!

How to Think About Mutable Objects – Variables "Point at" Objects

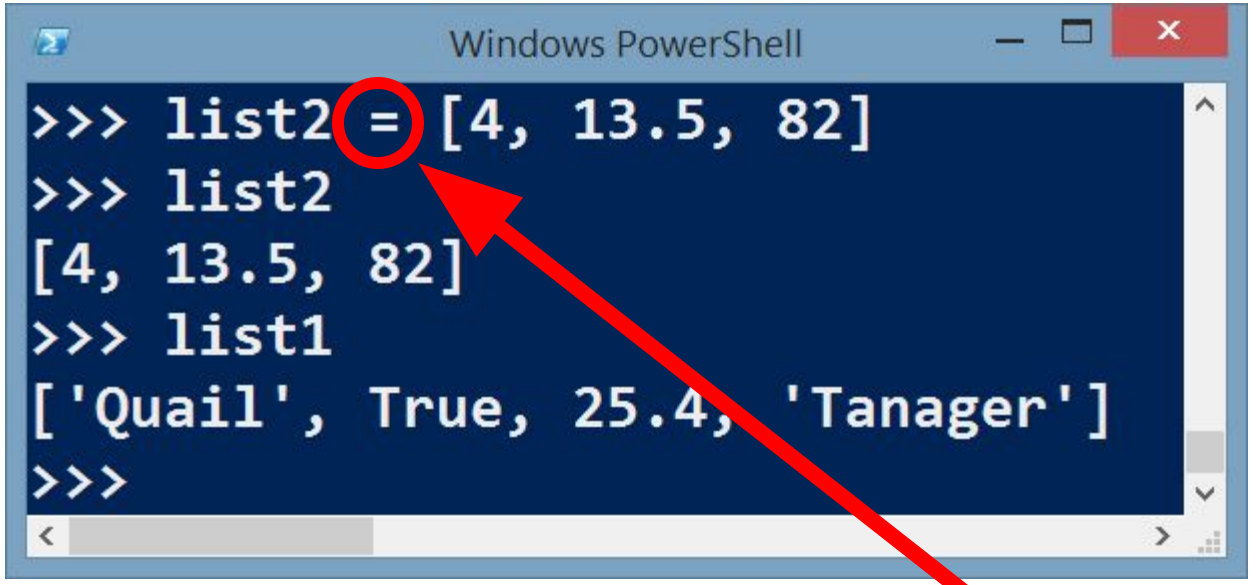
```
list1 = ["Quail", True, 25.4]
list2 = list1
```

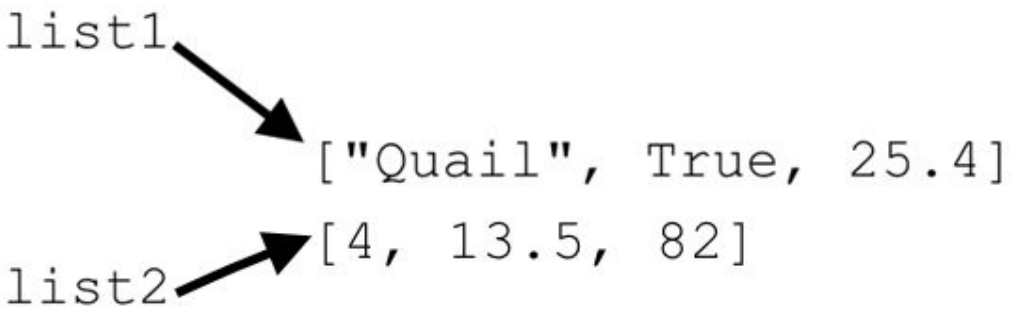


`list1` and `list2` represent the identical object in memory



Why Doesn't `list1` Change?





Assignment breaks the link between `list1` and `list2` – we are replacing not mutating in place

Copies of Lists

- You can create a *copy* of a list like this:

```
a = ["stew", "beer", "rabbit"]
b = a[:]  # creates a copy of a

print("a =", a)
print("b =", b) →
   a = ['stew', 'beer', 'rabbit']
   b = ['stew', 'beer', 'rabbit']
```

You can also use the `copy` *method:*

```
b = a.copy()
```

List Methods

Here are some more list methods:

- `list.append(value)`
- `list.insert(i, value)`
- `list.extend(other)`
- `list.count(value)`
- `list.index(value)`
- `list.pop([i])`
- `list.remove(value)`
- `list.clear()`
- `list.sort()`
- `list.reverse()`

- You should look at `help` on each of these to see more details:
  - e.g. `help(list.index)`
  - Try experimenting with each method
- See more about list methods here: `http://docs.python.org/3.3/`

Add Stuff to Lists: `append`, `insert`

```
food = []
```

- `append` adds an *element* to a list, placing it at the end

```
food.append("stew")
print(food) → ["stew"]
food.append("beer")
print(food) → ["stew", "beer"]
```

- `insert` adds an *element* to a list, placing it *before* the element at the given index

```
food.insert(0,"elk")
print(food) → ["elk", "stew", "beer"]
food.insert(2,"jello")
print(food) → ["elk", "stew", "jello", "beer"]
```

Method: `extend`

- `extend` adds a *list* of elements into a list (at the end)

```
food = ["stew", "beer", "cheese", "rabbit"]

drink = ["water", "beer", "wine"]

food.extend(drink)

print(food) →
   ["stew", "beer", "cheese", "rabbit", "water",
"beer", "wine"]

print(drink) → ["water", "beer", "wine"]
```

How else could we do this?

`append` vs. `extend`

- `append` takes an item as its argument and adds it to the end of the list. Puts ONE thing on.
- `extend` takes a list of items as its argument and adds all of the items to the end of the original list. Puts MANY things on.

Delete Stuff from Lists: `pop`, `clear`

- `pop` removes and returns the element at a given index (or the last element if no index is given)

```
food = ["stew", "beer", "cheese", "rabbit"]

item = food.pop(2)
print(food) → ["stew", "beer", "rabbit"]
print(item) → cheese

item = food.pop()
print(food) → ["stew", "beer"]
print(item) → rabbit
```

- `clear` removes all elements

```
food.clear()
print(food) → []
```

Delete Stuff from Lists : `remove`

- `remove(value)` removes the *first* occurrence of value

```
food = ["stew", "beer", "cheese", "beer"]
food.remove('beer')
print(food) → ["stew", "cheese", "beer"]

food.remove('beer')
print(food) → ["stew", "cheese"]

food.remove('beer')
ValueError:  list.remove(x):  x not in list
```

- How can we avoid this?

```
if 'beer' in food:
   food.remove('beer')
```

  - only remove it if you know it's in there

`pop` vs. `remove`

- `pop` takes an index as its argument or no argument at all. It deletes the item at that index, or the last item in the list (if no argument). Takes a NUMBER.
- `remove` searches for the argument and deletes the first occurrence of it. Takes an ITEM.

Find Stuff in Lists: `count`, `index`

- `count(value)`: return number of occurrences of value

```
food = ["stew", "beer", "cheese", "beer"]
print(food.count('beer'))  → 2
print(food.count('beef'))  → 0
```

- `index(value)`: return first index of value

```
food = ["stew", "beer", "cheese", "beer"]
print(food.index('beer'))  → 1
```

```
food = ["stew", "beer", "cheese", "beer"]
print(food.index('beef'))  →
ValueError:  'beef' is not in list
```

- How can we avoid this?

`in` operator

- The **in** operator works on lists
  - Tests if a given *element* is in a list

```
food = ["apples", "beer", "cheese", "duck"]

print("beer" in food) → True

print("earwax" in food) → False

print("Beer" in food) → False

print("Beer".lower() in food) → True
```

Concatenating

- Recall that concatenating two *strings* using the + operator results in a *new* string
- You can concatenate *lists* with the + operator:

```
food = ["stew", "bread", "cheese", "duck"]
drink = ["juice", "beer", "water"]

meal = food + drink
print(meal) →
['stew', 'bread', 'cheese', 'duck', 'juice',
'beer', 'water']
```

- Concatenating two lists results in a *new* list

Lists are Mutable

- Recall that strings are *immutable*:

```
string = "hello"
string[0] = "X"
TypeError: 'str' object does not support item
assignment
```

- Unlike strings, lists *are* mutable
- You can change an element by *assigning* to an index.

Lists are Mutable

```
food = ["apples", "beer", "cheese", "duck"]

print(food) →
   ["apples", "beer", "cheese", "duck"]

food[0] = "stew"

print(food) →
   ["stew", "beer", "cheese", "duck"]

food[-1] = "rabbit"

print(food) →
   ["stew", "beer", "cheese", "rabbit"]
```

Lists

```
food = ["apples", "beer", "cheese", "duck"]
```

- What about that `food` variable?
  - Is it somehow special or magical?
- Nope, it's just a variable, so assigning to it like this:

```
food = 99
```

- makes `food` refer to `99` and sends our list down to Python garbage collection never to be seen again... :(

Traversing Lists by Index

- Since lists are mutable you can do things like this:

```
food = ['apples', 'beer', 'cheese', 'duck']
for i in range(len(food)):
   food[i] = food[i].upper()
print(food)
```

Traversing Lists by Index

- Try changing the code to this:

```
food = ['apples', 'beer', 'cheese', 'duck']
for i in range(len(food)):
    food[i] = food[i].upper()
    print(food)
```

Traversing Lists by Element

- What happens when we do this:

```
food = ['apples', 'beer', 'cheese', 'duck']
for item in food:
    item = item.upper()
print(food)
print(item)
```

Traversing Lists by Element

- If we did this:

```
food = ['apples', 'beer', 'cheese', 'duck']
item = 'apples'
item = item.upper()
```

- Would you expect that to change the list?
  - Nope, we're just changing the value that `item` refers to